

Jake DeMuesy

CS 499

6/28/26

Professional Self-Assessment

I entered the software engineering industry in 2016 as a self-taught developer. I worked well enough to land a junior role and eventually get to a Software Engineer II position, but I always knew the degree was the gap I'd have to close eventually to reach seniority. Completing the Computer Science program at SNHU while working full-time has been the practical way to do that. This ePortfolio is the result of that overlap -- work I've done in a structured academic environment, informed by what I've seen and built in production systems at Fortune 500 and Fortune 100 companies over the last decade.

What I'm targeting next is a move into either finance or aerospace. I already have experience in the finance space from my current work, and .NET demand in aerospace -- particularly in the Denver area -- has been on my radar lately. The degree makes both paths more realistic, and the capstone gave me a portfolio I can actually walk through in an interview.

Program Experience and Skill Development

The area the program has probably pushed me the most on is communicating with non-technical stakeholders. My day-to-day as a software engineer involves translating technical decisions for product managers, compliance teams, and sometimes clients. But CS 499 formalized that in a way that actual work doesn't. Writing narratives for each enhancement meant I had to explain not just what I built, but why a specific architectural decision mattered and what trade-off it was solving. The code review assignment earlier in the capstone required me to walk through weaknesses in my own code out loud, which is a different skill than writing about them. Being able to articulate those things clearly -- to a technical reviewer and a general audience at the same time -- is something I'll use directly in interviews and in day-to-day stakeholder communication.

Collaborating in team environments is something I've had to develop mostly outside of academic coursework, since most class projects are individual. What SNHU's program gave me there was the discussion board structure across courses, which forces you to read and respond to how other students are approaching the same problem. In IDS-405 specifically, working through the AFL-CIO labor movement research involved peer review cycles and structured response posts where I had to engage seriously with approaches that differed from mine. That kind of asynchronous, written exchange is actually pretty close to how a lot of real code review and design discussion happens in distributed teams. In my actual job, I've worked across multiple cross-functional teams and have had to get alignment on technical decisions with people who have

competing priorities -- that experience plus the program's communication emphasis together give me a clearer sense of how to manage those situations.

On data structures and algorithms, the program covered a lot of ground -- from the MNIST classification and CartPole work in CS 370 to the full deep Q-learning implementation in the pirate maze project. What clicked for me during that course was understanding the difference between an algorithm that works and one that's been designed with the right trade-offs in mind. Uniform random sampling in a replay buffer works, but prioritized experience replay works better by directing training attention toward transitions with high TD error. The background I already had in production systems helped me evaluate those trade-offs with a practical lens, but CS 370 gave me the formal vocabulary to articulate them.

Software engineering got the most direct overlap with my professional experience. I've been writing .NET applications in production environments for years, and the CS 465 MEAN stack course was interesting specifically because it forced me outside that comfort zone -- Node.js, MongoDB, Express, Angular, all at once. Getting a working full-stack application running in an unfamiliar stack teaches you something about what actually transfers between languages and frameworks versus what's specific to the tool. The CS 330 graphics work was the furthest from anything I'd done professionally -- C++ and OpenGL are not part of my day-to-day -- but the architectural patterns I applied to the capstone enhancement (composite scene graph, SRP enforcement, dependency injection via RenderContext) are the same patterns I'd apply in any production codebase. That's the transferability piece the capstone was designed to demonstrate.

Security is an area where my professional background and the program have intersected most directly. I've worked in environments where access control and data protection are compliance requirements, not optional considerations. CS 340 brought that into the academic work in a concrete way -- implementing role-based access control in the CRUD module wasn't just a checkbox; it required thinking through what the analyst and admin roles actually need to do, where the enforcement should live (application layer vs database layer), and why defense in depth matters when either of those can fail independently. That kind of security reasoning is something I've had to apply in real systems and it was good to formalize it through the coursework.

Portfolio Artifacts Overview

The three artifacts in this portfolio cover software engineering and design, algorithms and data structures, and databases -- the three categories the capstone requires. They're not randomly chosen; each one targets a specific gap between what the original coursework demonstrated and what I'd want a technical reviewer to see in a portfolio.

The first artifact is the CS 330 OpenGL 3D scene in C++, representing software engineering and design. The original version was a functional but flat piece of academic work -- a big source file doing everything at once. The enhanced version refactored it into a proper class hierarchy: a composite scene graph with `SceneNode` and `RenderableObject`, a `ShaderProgram` abstraction with a uniform location cache, a `LightingConfig` file parser for external light definitions, and a `TextureManager` class extracted from `SceneManager` to enforce the Single Responsibility Principle. The comments throughout were updated to explain the engineering reasoning behind decisions, not just describe what the code does. I believe this artifact shows C++ object-oriented design, separation of concerns, and the ability to take a working but unstructured piece of code and make it maintainable.

The second artifact is the CS 370 deep Q-learning pirate maze agent in Python, representing algorithms and data structures. The original submission was a Jupyter notebook with the Q-training loop as the core student deliverable. The enhanced version modularized that into discrete Python files: `TreasureMaze.py` as the environment state machine, `GameExperience.py` as the session orchestrator with prioritized experience replay, `ParameterTuner.py` as a systematic hyperparameter search tool that logs results to a CSV, and `test_parameter_tuner.py` as a validation suite that runs without a live training loop. A legitimate bug was also found and fixed -- a duplicate `__init__` method in `GameExperience.py` was silently overwriting the first constructor, which meant the entire PER system was broken despite the code looking correct. I believe that this artifact shows reinforcement learning implementation, modular Python design, empirical testing methodology, and the ability to catch subtle behavioral bugs through code review.

The third artifact is the CS 340 Grazioso Salvare Animal Rescue Dashboard in Python and MongoDB, representing databases. The original version was a working Dash dashboard with a basic CRUD module and client-side data processing. The enhanced version added role-based access control with distinct admin and analyst roles enforced at both the application and database level, a compound index strategy targeting the actual rescue-type filter fields in the correct predicate order (equality fields first, range field last), a server-side aggregation layer in a separate `animal_analytics.py` module with MongoDB pipeline methods for breed counts, outcome distributions, and time-bucketed trends, a shared query builder function so the table filter and aggregation chart can't drift apart, and a test suite for the analytics module that simulates MongoDB's aggregate pipeline in Python without requiring a live connection. This artifact shows database engineering, query optimization with empirical verification, security through least-

privilege access design, and separation of concerns between a data access layer and an analytical reporting layer.

Together the three artifacts cover different languages (C++, Python), different paradigms (graphics, machine learning, web applications), and different engineering concerns (architecture, algorithmic design, data management and security). That range is intentional -- it's a more accurate picture of what I can do than any single project would be.